

Browser, CDN, Application, and Database Layers

 systemdesignschool.io/fundamentals/caching-tiers



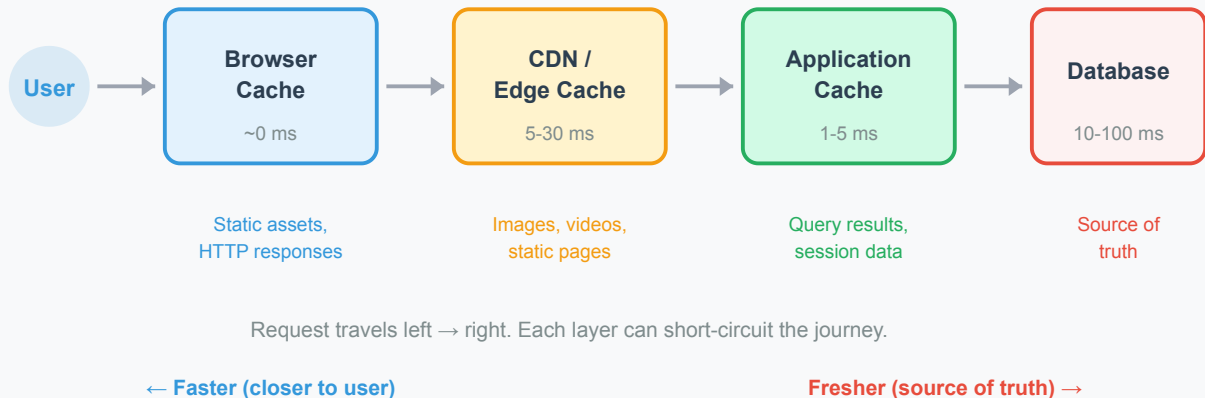
The Multi-Layer Defense

The previous article established caching as time-bounded copies stored closer to the requester. But "closer" can mean different things. A copy on the user's device is closer than one in a nearby data center, which is closer than one on the origin server. Each layer intercepts requests at a different point, and each trades freshness for a different amount of speed.

The Four Layers

A request travels through up to four caching layers before reaching the database. Each layer can short-circuit the journey by returning a cached response.

Caching Tiers: From User to Database



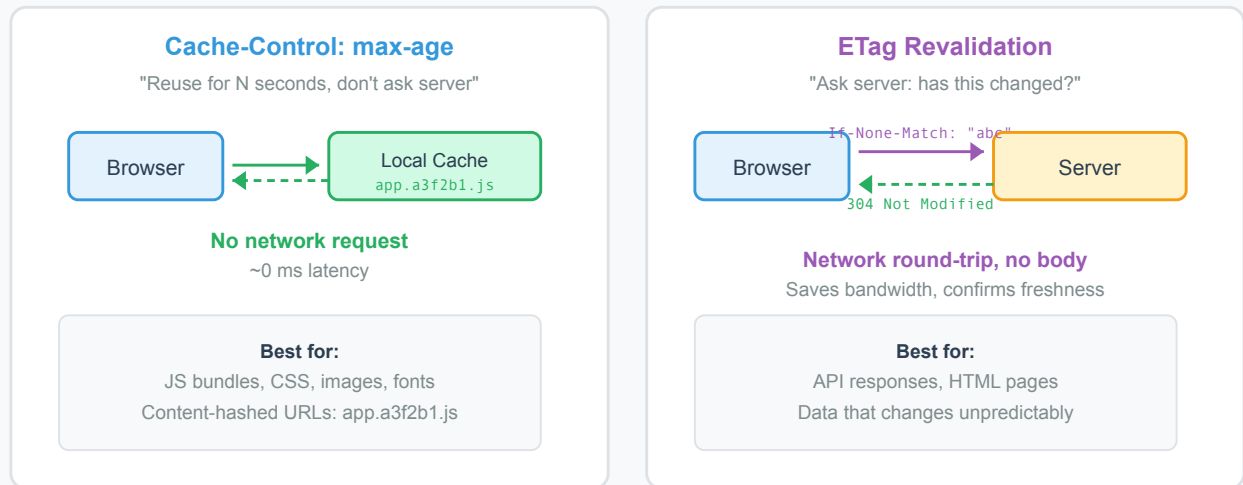
The layers closest to the user are fastest but hardest to invalidate. The layers closest to the database are freshest but add the most latency. Understanding which layer to use—and which to skip—is a core system design skill.

Browser Cache

The browser stores HTTP responses locally on the user's device. When a user revisits a page, the browser checks its local cache before making any network request. A cache hit costs zero network latency.

The server controls browser caching through HTTP headers. The `Cache-Control` header specifies how long a response stays valid. A header like `Cache-Control: max-age=3600` tells the browser to reuse the response for one hour without contacting the server.

Browser Cache: Two Strategies



Browser caching works best for **static assets**: JavaScript bundles, CSS files, images, and fonts. These change only during deployments. A version hash in the URL (e.g., `app.a3f2b1.js`) lets you cache aggressively—set max-age to a year, and the filename changes when content changes.

Browser caching is trickier for dynamic data. A user's notification count changes every few seconds. The shopping cart updates on every add. Short TTLs or ETag revalidation (where the browser asks "has this changed?" and the server responds with a lightweight 304 Not Modified) can help, but for rapidly changing state the request typically travels deeper.

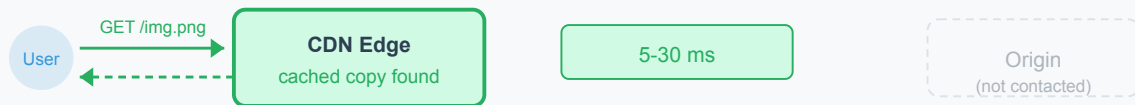
CDN / Edge Cache

A **CDN** (Content Delivery Network) stores content on servers distributed across geographic regions. When a user in Tokyo requests an image, the CDN serves it from a Tokyo edge server instead of the origin in Virginia. Latency drops from roughly 150ms to under 10ms.

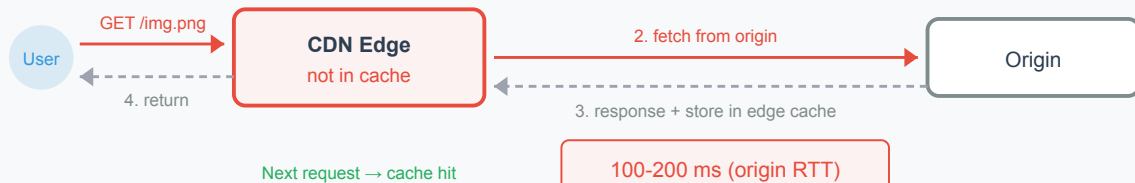
CDNs cache at the HTTP layer. When an edge server receives a request it hasn't seen, it fetches from the origin, stores the response, and serves future requests locally. The origin sets caching rules through the same `Cache-Control` headers the browser uses.

CDN Edge: Cache Hit vs Cache Miss

Hit: Edge has content



Miss: Edge fetches from origin



CDNs excel at **static content at scale**: images, videos, downloadable files, and pre-rendered pages. They also handle traffic spikes—edge servers can absorb the vast majority of requests when hit rates are high.

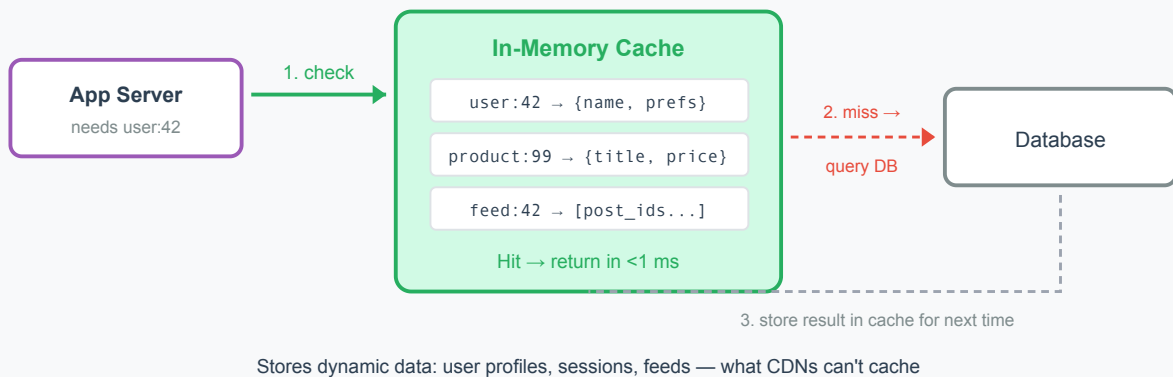
CDNs struggle with **personalized or rapidly changing content**. A page that shows the user's name, cart items, or real-time prices can't be cached at the CDN without careful partitioning. That boundary becomes clearer in the CDN vs application cache discussion.

Application Cache

The **application cache** is an in-memory data store (like Redis or Memcached) that sits between the application servers and the database. This is the layer most relevant to system design interviews.

When an application server needs data, it checks the cache first. On a hit, it reads from memory—typically sub-millisecond to a few milliseconds over the network. On a miss, it queries the database, stores the result in cache, and returns it. Subsequent requests for the same data hit the cache.

Application Cache: Check Cache Before Database

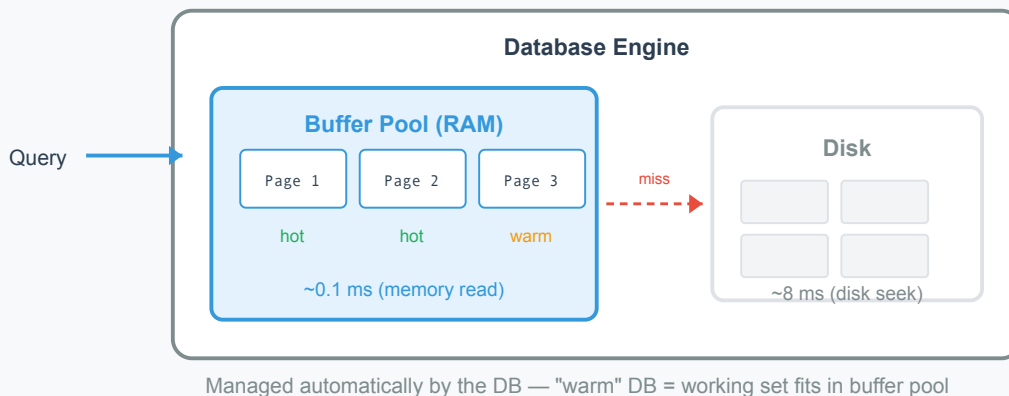


Application caching handles what CDNs struggle with: **dynamic, personalized data** with complex access patterns. User profiles, product details, session state, feed timelines, search results—all fit naturally into an application cache.

The application cache is also the layer you control most precisely. You decide what to cache, when to expire it, and how to invalidate it on writes. This flexibility comes with complexity. The articles on read patterns, write patterns, and consistency explore these decisions in depth.

Database Buffers

Database Buffer Pool: The Invisible Cache



Databases maintain their own internal caches. A relational database keeps frequently accessed pages in a **buffer pool** (a region of RAM). When a query touches data already in the buffer pool, it reads from memory rather than disk.

This layer is largely invisible to application developers. The database manages it automatically. But it matters for capacity planning—a database with enough RAM to hold its working set in the buffer pool performs dramatically better than one that constantly reads from disk.

You rarely configure this layer in system design interviews. But understanding it explains why a "cold" database (just restarted, empty buffer pool) is slower than a "warm" one, and why cache failures that redirect traffic to the database cause cascading slowdowns.

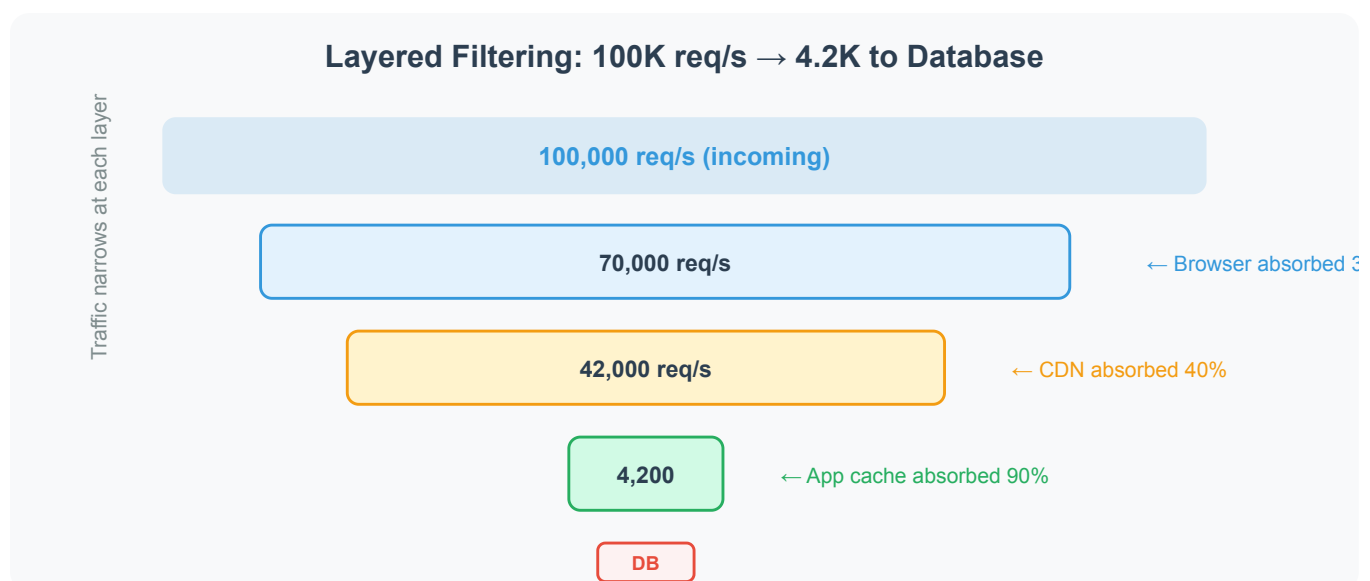
Layers Compose

In production, multiple layers work together. A typical web request might proceed:

1. Browser checks its local cache → miss (first visit)
2. CDN edge server checks its cache → hit for static assets, miss for personalized API calls
3. Application server checks the in-memory cache → hit for popular data
4. Database query (only if all layers miss) → reads from buffer pool or disk

Each layer reduces the load on layers below it. The browser absorbs repeat visits. The CDN absorbs geographic distribution. The application cache absorbs hot data. The database handles only cold or freshly written data.

The power of layered caching is multiplicative. If the browser catches 30% of requests, the CDN catches 40% of what remains, and the application cache catches 90% of what passes through—the database sees only 4.2% of original traffic.



Choosing the Right Layer

Not every system needs all four layers. The choice depends on your data and access patterns.

Static, public, rarely changing: Browser + CDN. Marketing pages, product images, documentation.

Dynamic, read-heavy, tolerates seconds of staleness: Application cache. User profiles, product catalogs, feed rankings.

Dynamic, user-specific, low latency required: Application cache with short TTL (time-to-live). Session data, notification counts, dashboard metrics.

Frequently changing, consistency-critical: Skip caching or use very short TTL with active invalidation. Inventory counts, account balances, real-time bidding.

From there, the boundary between CDN caching and application caching becomes the central architectural decision for read-heavy systems.